TRAMA · PRESENTACIÓN FINAL

Proyecto MANI

Arquitectura, datos, diseño, infraestructura, pruebas y cierre

SAD · DD · SDD · Infraestructura · Pruebas · Acta de cierre

5 de noviembre de 2026 · v1.0.4 en producción · Arquitectura de Software 2026-03 · Pontificia Universidad Javeriana

AGENDA

De qué vamos a hablar

01	**Contexto**	Qué es MANI y qué problema resuelve
02	**SAD**	Arquitectura: qué guía las decisiones
03	**DD**	Datos: cómo se guarda y protege la información
04	**SDD**	Diseño: piezas y decisiones detalladas
05	**Infraestructura**	Dónde y cómo corre MANI
06	**Pruebas**	Qué comprobamos y con qué resultado
07	**Cierre**	Avance, acta, pendientes y lecciones

CONTEXTO

MANI: formalizar servicios técnicos

El problema

Las empresas de servicios coordinan técnicos y clientes por WhatsApp y llamadas. No verifican a los técnicos, una solicitud puede quedar con dos técnicos y no hay precios de referencia.

La solución

Una plataforma SaaS multi-tenant: cada empresa (tenant) usa MANI como su propia app, con sus datos aislados y su propia configuración.

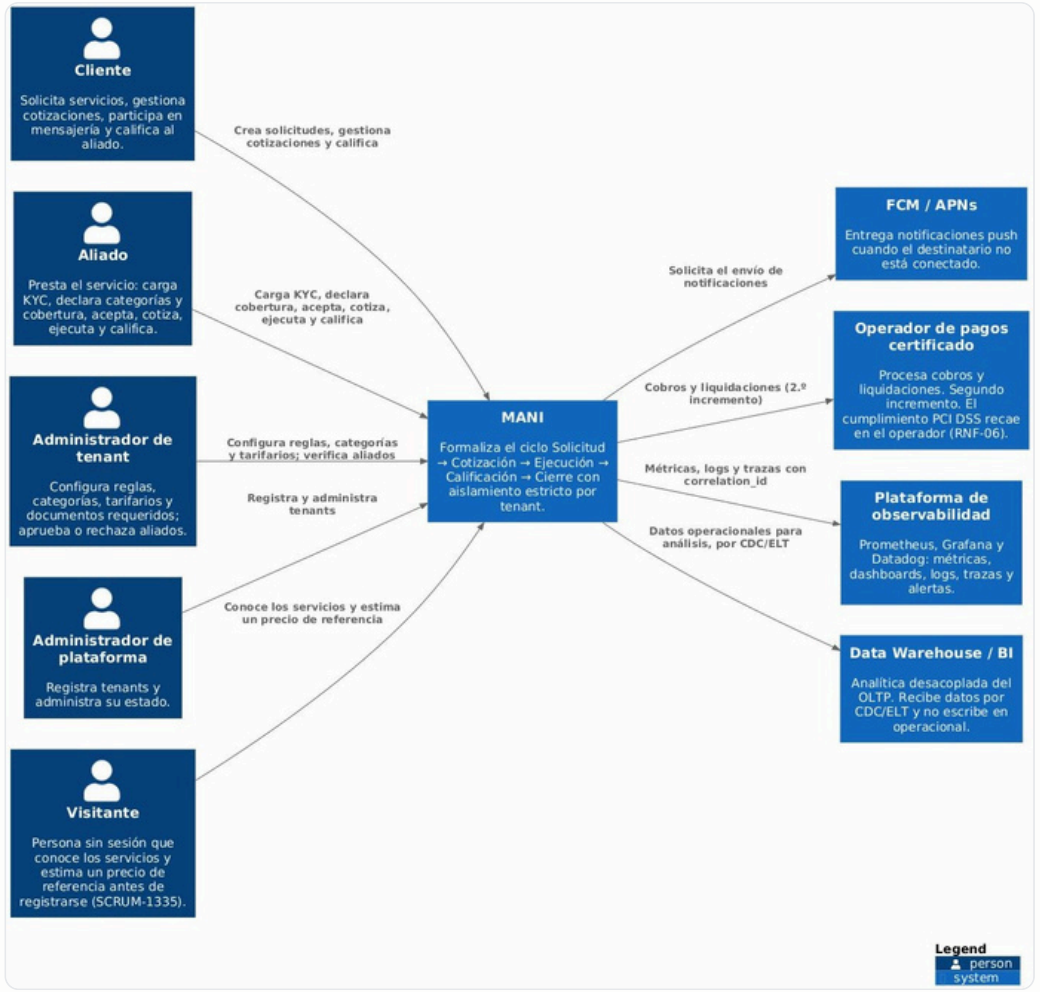
EL CICLO DEL SERVICIO QUE CUBRE MANI

Solicitud → **Cotización** → **Ejecución** → **Calificación** → **Cierre**

En pocas palabras: una sola app sirve a muchas empresas, pero ninguna ve los datos de otra.

CONTEXTO · C4 NIVEL 1

MANI, sus actores y sus vecinos



5

actores

Visitantesin sesión, cliente, aliado, administrador de tenant y administrador de plataforma.

4

sistemas externos

Notificaciones push(FCM/APNs), operador de pagos certificado, observabilidad y Data Warehouse.

En pocas palabras: MANI cubre el ciclo Solicitud → Cotización → Ejecución → Calificación → Cierre; la analítica sale por CDC/ELT y nunca escribe en la base operacional.

Lo que guía la arquitectura

Un **driver** es un requisito tan importante que obliga a tomar decisiones de arquitectura.

RF-01 · Crítica

Aislamiento entre empresas

Ninguna empresa puede ver los datos de otra.

RF-02 · Crítica

Configurar sin programar

Cada empresa ajusta reglas, categorías y tarifas sin un nuevo despliegue.

RF-14 · Crítica

Una sola asignación

Si dos técnicos aceptan a la vez, solo uno gana.

RESTRICCIONES QUE NO NEGOCIAMOS

- Cobertura por zonas, no por GPS (ADR-0011)
- Pagos con operador certificado: PCI DSS es del operador
- Backend en Java y .NET, por contrato (ADR-0019)

Cuatro capas y una sola puerta de entrada

Presentación	Flutter: app web y móvil	Separar interfaz · Mantenibilidad
Borde	Gateway NGINX: valida el token y enruta	Una sola entrada · Seguridad
Negocio	Core (Node.js) · Rules (Java) · Dispatch (.NET)	Dividir por capacidad · Flexibilidad
Datos	Supabase: PostgreSQL con RLS, Auth, Storage	Defensa por empresa · Seguridad
Transversal	Eventos, circuit breaker y contratos OpenAPI	Desacoplar · Fiabilidad

En pocas palabras: la app nunca habla directo con la base de datos: todo pasa por el Gateway.

Cinco atributos, cada uno con su métrica

Basados en la norma ISO/IEC 25010. Son los de prioridad 1.

1

Seguridad

100 % de endpoints privados con autenticación y 0 accesos entre empresas.

2

Fiabilidad

APIs críticas disponibles ≥ 99,9 % y 0 dobles asignaciones.

3

Eficiencia

Meta: p95 ≤ 500 ms con 300 sesiones. Bajo carga no cumple: se agota el pool de Core.

4

Mantenibilidad

Cobertura de código nuevo ≥ 80 %. Core 90,1 %, Rules 92,3 %.

5

Flexibilidad

La misma imagen pasa entre ambientes y escala de 2 a 6 réplicas.

En pocas palabras: cada atributo tiene un número para comprobar si se cumple; los umbrales viven en el SDD y los resultados, en la diapositiva de pruebas.

Lo que ganamos y lo que cedimos

Varios servicios en distintos lenguajes

Ganamos: mantener y escalar cada uno por separado.

Cedimos: más costo para operar cinco servicios.

Gateway como única entrada

Ganamos: seguridad y políticas en un solo punto.

Cedimos: un salto de red más, que medimos con el p95.

Lógica en servicios, no en la base

Ganamos: código que se prueba y se versiona.

Cedimos: reescribir unas 32 funciones PL/pgSQL.

Un esquema de datos por dominio

Ganamos: límites claros entre servicios.

Cedimos: consultas cruzadas más costosas.

ADR clave: **ADR-0012** Supabase con RLS · **ADR-0019** Java y .NET · **ADR-0029** Kafka para eventos · **ADR-0031** Tokens ES256

Un esquema por dominio, cada uno con un dueño

1. **Cada servicio escribe solo en su esquema**
   Ningún servicio lee ni modifica tablas de otro dominio.

2. **Cada tabla lleva tenant_id y RLS**
   La propia base de datos impide ver filas de otra empresa (ADR-0012).

3. **Entre esquemas, referencias lógicas**
   Sin lectura cruzada; la analítica va en un Data Warehouse aparte.

En pocas palabras: RLS (Row-Level Security) es un filtro dentro de la base de datos que solo deja ver las filas de tu empresa.

Cómo se protegen y publican los datos

2

tablas outbox

servicio.outbox (Core) y despacho.outbox (Dispatch). El evento se guarda en la misma transacción que el dato: no se pierde ni se duplica.

6

roles de base de datos

Un rol por servicio, sin superusuario ni BYPASSRLS. Ningún servicio lee un esquema ajeno.

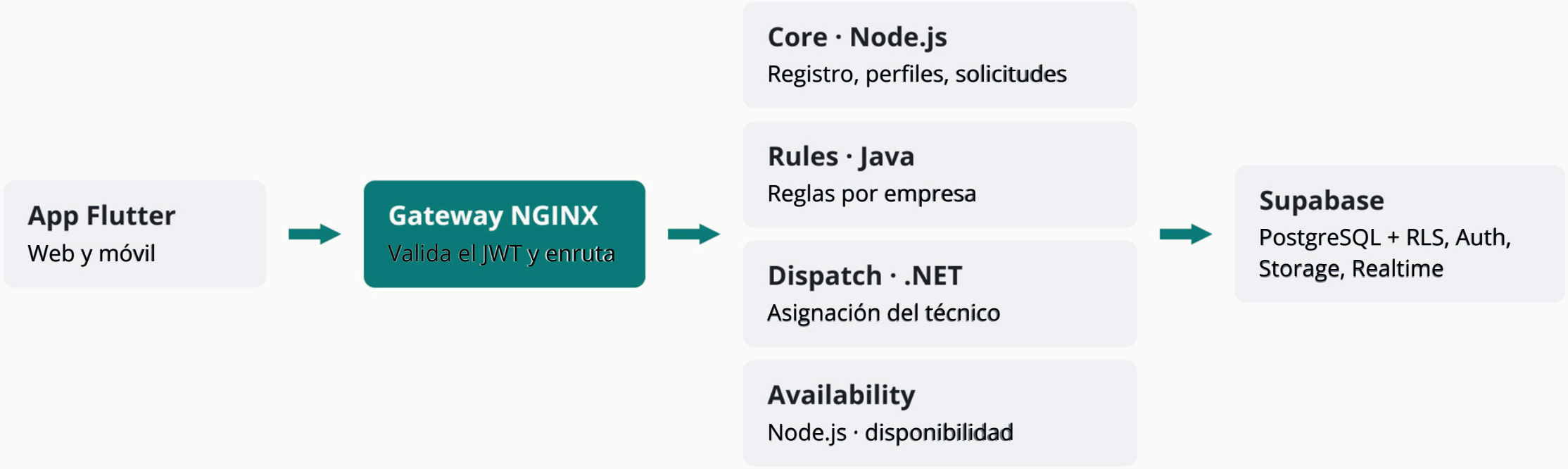
25

políticas RLS

Sobre 8 migraciones versionadas en Supabase: la base se reconstruye igual en cada ambiente.

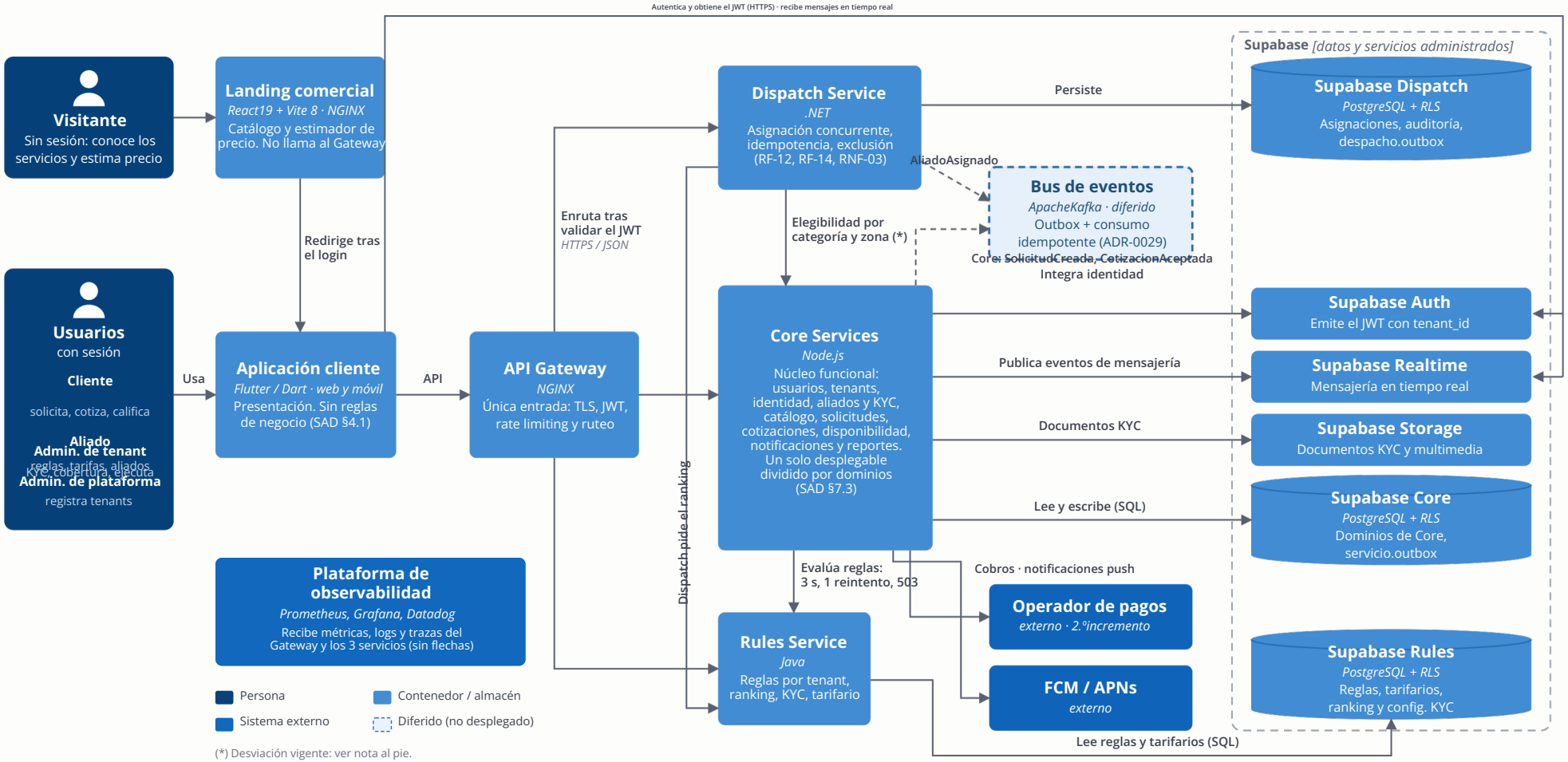
Punto abierto que declaramos: la tabla solicitud vive en el esquema de Dispatch, pero la crea Core. Moverla toca tablas ya creadas, así que quedó documentada como deuda.

Qué piezas tiene MANI y cómo se hablan

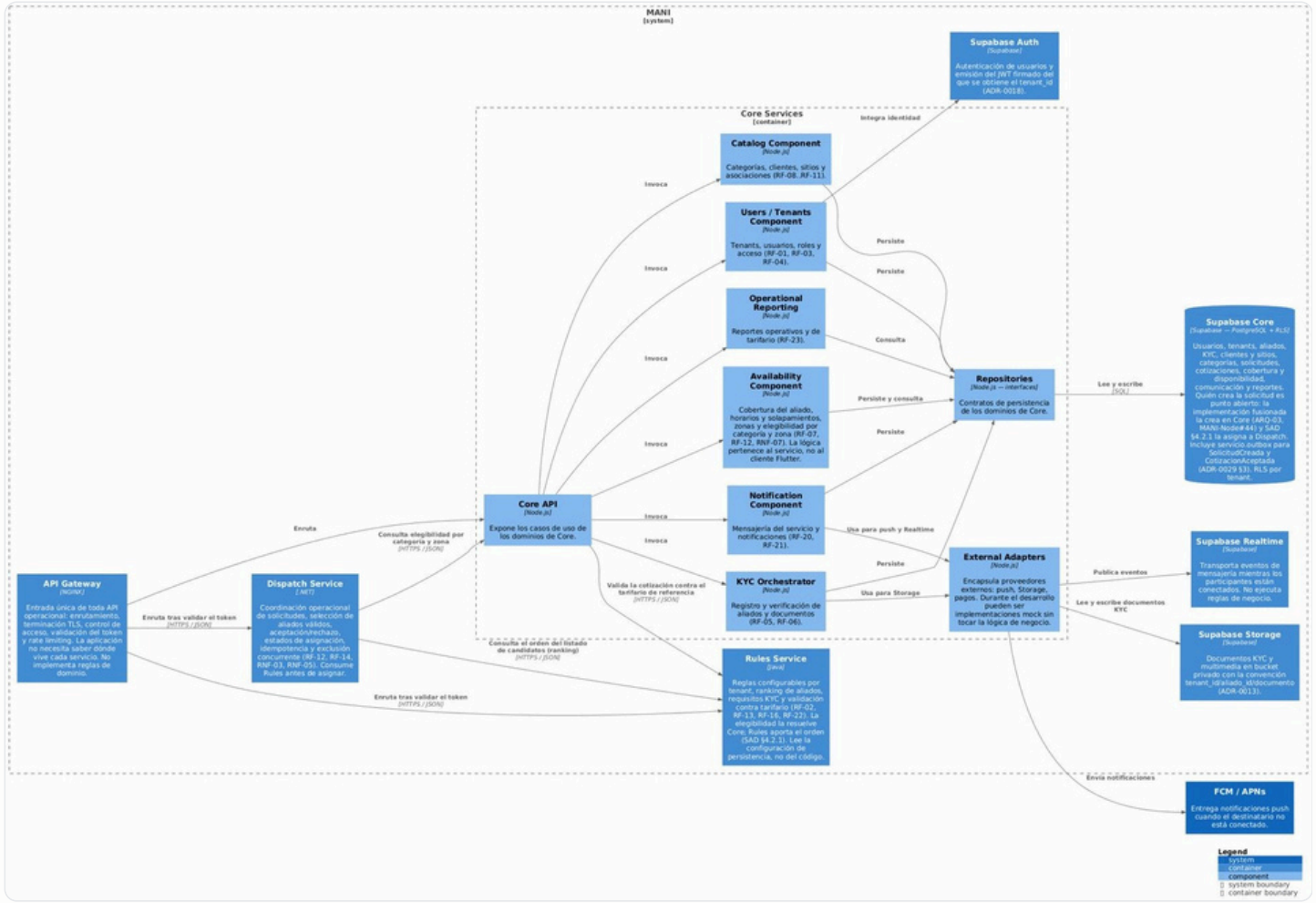


En pocas palabras: C4 muestra el sistema por niveles de zoom; esta es la vista de contenedores.

Contenedores: qué se despliega y dónde guarda



Core por dentro: un desplegable, nueve componentes



Core API

Única puerta del contenedor: expone los casos de uso de cada dominio.

Availability vive aquí

Cobertura, horariosyelegibilidad por categoría y zona: lógica del servicio, no del cliente Flutter.

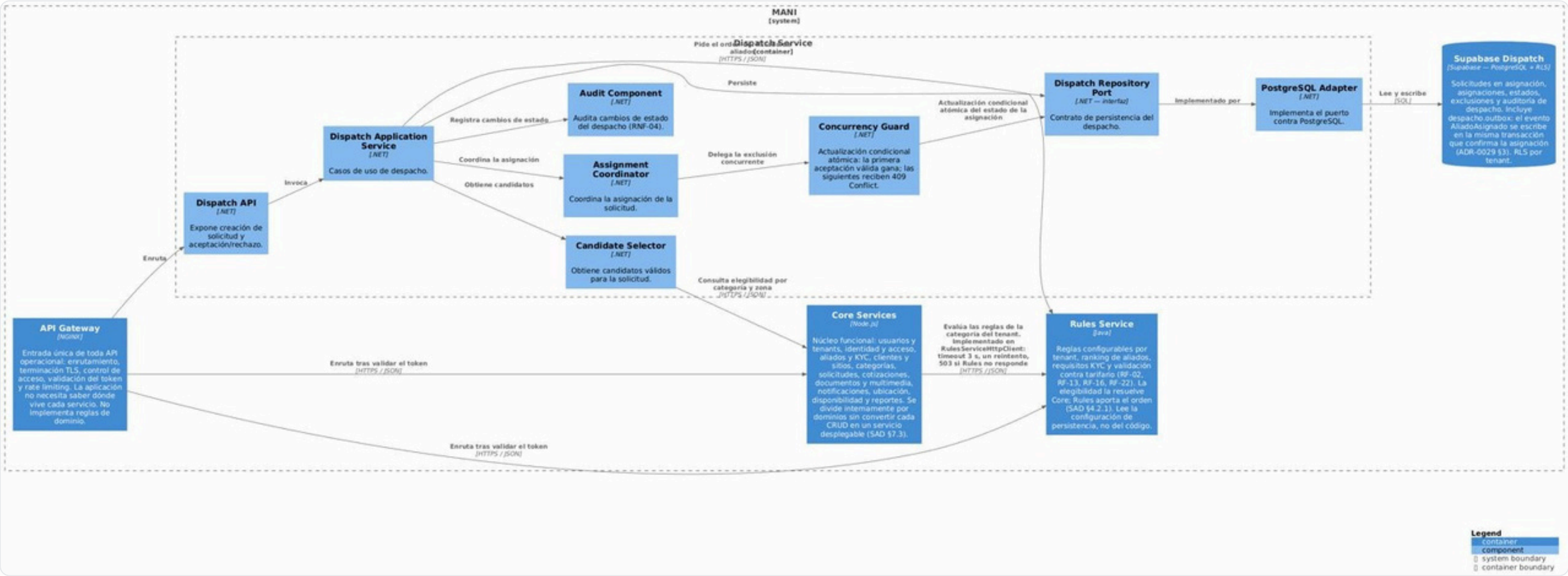
Repositories

Únicoacceso aSupabase Core; ningún componente escribe SQL por su cuenta.

External Adapters

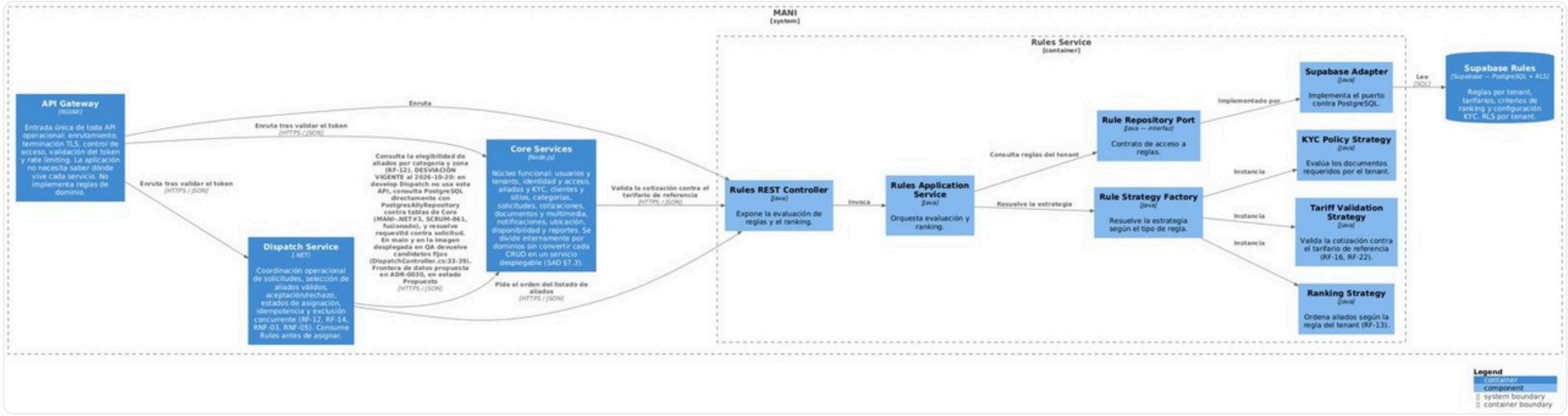
Push,Storage y pagos detrás de una interfaz: en desarrollo se usan mocks.

Dispatch por dentro: una sola asignación



En pocas palabras: la exclusión concurrente vive en **Concurrency Guard**: una actualización condicional atómica hace que la primera aceptación válida gane y las siguientes reciban 409 Conflict (RF-14).

Rules por dentro: una estrategia por tipo de regla



KYC Policy

Quédocumentos exige cada tenant al aliado.

Tariff Validation

Validalacotización contra el tarifario de referencia (RF-16, RF-22).

Ranking

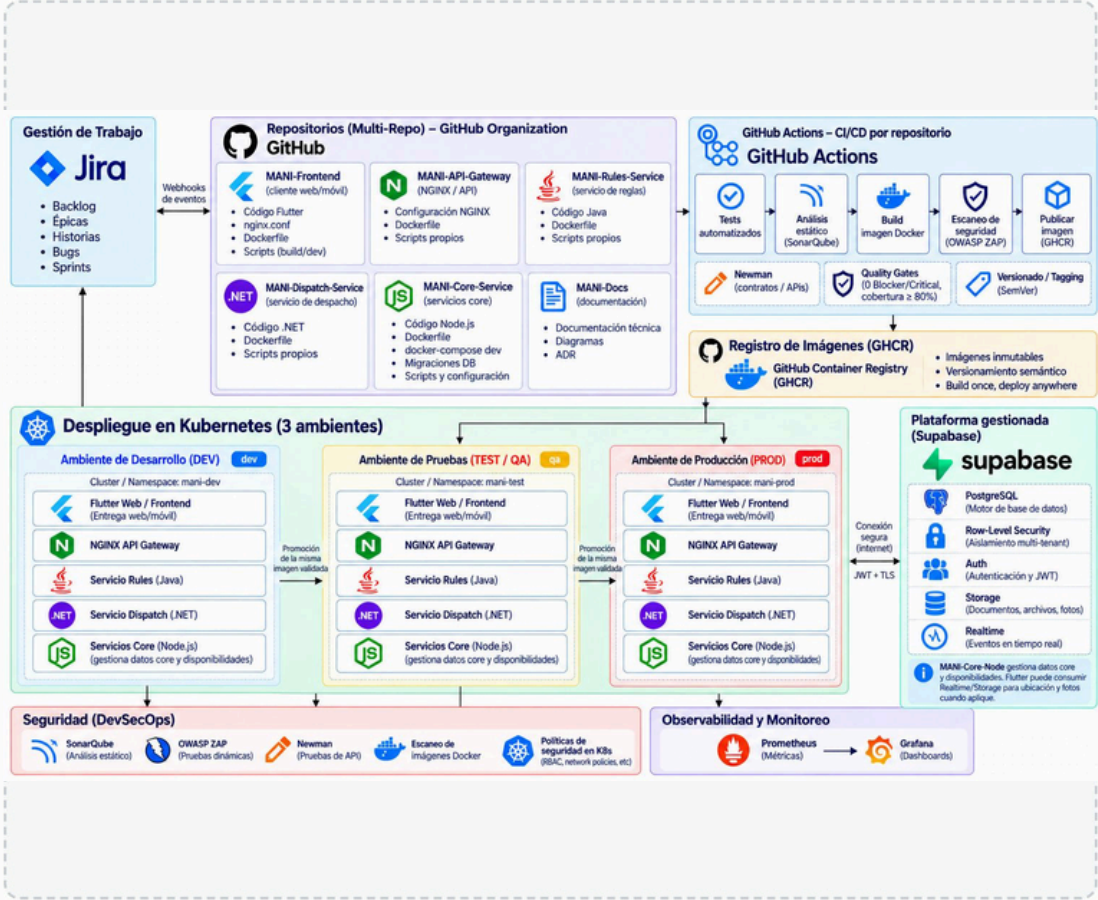
Ordena a los aliados candidatos según la regla del tenant (RF-13).

En pocas palabras: Rules lee la configuración de cada empresa desde su base, no del código: por eso las reglas cambian sin desplegar (RF-02).

Seis decisiones de diseño

Tema	Decisión
Integración hoy	HTTP sincrónico por red interna. Si un servicio no responde, se falla cerrado (503).
Integración final	Bus de eventos con Apache Kafka (ADR-0029).
Eventos	SolicitudCreada y CotizacionAceptada (Core); AliadoAsignado (Dispatch).
Publicación	Transactional outbox, obligatorio.
Consumo	Idempotente por event_id: recibir dos veces no duplica nada.
Base de datos	Una sola serie de migraciones, a cargo de Core.

Dónde y cómo corre MANI



Ambientes

QA en una máquina de la red Javeriana (por VPN): app y API en el mismo puerto, la API bajo /api/. Producción desde el 5-nov con la v1.0.4.

Contenedores

Un contenedor por pieza con Docker Compose: front Flutter web, Gateway NGINX, Core, Rules, Dispatch y Availability.

Servicios gestionados

Supabase (PostgreSQL con RLS, Auth y Storage) y Kafka para los eventos (ADR-0029).

Cómo llega un cambio

Pull request a develop → revisión → despliegue en QA → validación del PO → producción.

INFRAESTRUCTURA · DESPLIEGUE

Producción el 5 de noviembre

v1.0.4
release en producción
Etiqueta en los 7 repositorios

200
todos los servicios
Gateway, Core, Rules, Dispatch, Flutter y Landing

1
fuente de migraciones
Serie única en MANI-Node (ADR-0030)

:staging → :latest
promoción de imágenes
La misma imagen validada en QA

2
ambientes separados
QA y producción, cada uno con su Supabase

5/11
fecha planeada
Despliegue en la fecha comprometida

DOCUMENTO DE PRUEBAS (TD)

Qué comprobamos con pruebas

1 237
pruebas automáticas en verde
Pruebas unitarias y de integración en los repositorios

806 de 847
casos por API pasan (95 %)
Corrida de QA del 5-nov con los roles nuevos de base de datos

75
historias con matriz de QA
70 finalizadas al corte de QA; ninguna falla nueva por API

53
pruebas de la PO desde la app
3-4 nov: 39 funcionan; las fallas eran pantallas sin conectar, ya corregidas

90,1 % · 92,3 %
cobertura de Core y Rules
Flutter: 88,6 % de código nuevo

5
bugs abiertos por QA
1633 a 1637: pago tras finalizar, foto, aliados, estado en vivo y cliente-tenant

MANI · TRAMA · Fuente: Reporte de QA del Sprint 5, corte 2 (Santiago Hernández, 5-nov)

DOCUMENTO DE PRUEBAS · ESCENARIOS DE CALIDAD

45 escenarios medidos al cierre

QA midió los **45 escenarios** del SDD sobre la versión del 5 de noviembre.



Ninguno cumple completo porque en cada uno falta alguna parte del método del SDD (por ejemplo, Kubernetes, inyección de fallas o pruebas de carga ajustadas).

Aislamiento entre empresas (QAS-01)
Parcial: 6 de 6 accesos cruzados rechazados por API; falta repetir con los roles nuevos de base de datos.

Una sola asignación (QAS-41)
Parcial: nunca hay doble asignación, pero en una carrera de 11 aceptaciones, 2 dan error 503.

Rendimiento bajo carga (QAS-03, 15, 19)
No cumple: p95 de 3 s y errores a 50 req/s. Causa: se agota el pool de conexiones de Core (5 conexiones).

Trazabilidad y seguridad del borde (QAS-14, 22, 43)
No cumple: sin rate limiting, Dispatch no registra el correlation ID y no hay TLS en QA.

RESULTADO · SPRINT 5

El sprint en cifras

84 de 85
historias en Done
Pendiente: 1383, cifrado de datos sensibles (reabierta)

532 SP
completados
De 537 SP en el sprint: 99,1 %

123 de 124
ítems finalizados
85 historias, 30 tareas técnicas y 9 bugs

9 de 9
defectos corregidos
Todos registrados y cerrados en el sprint

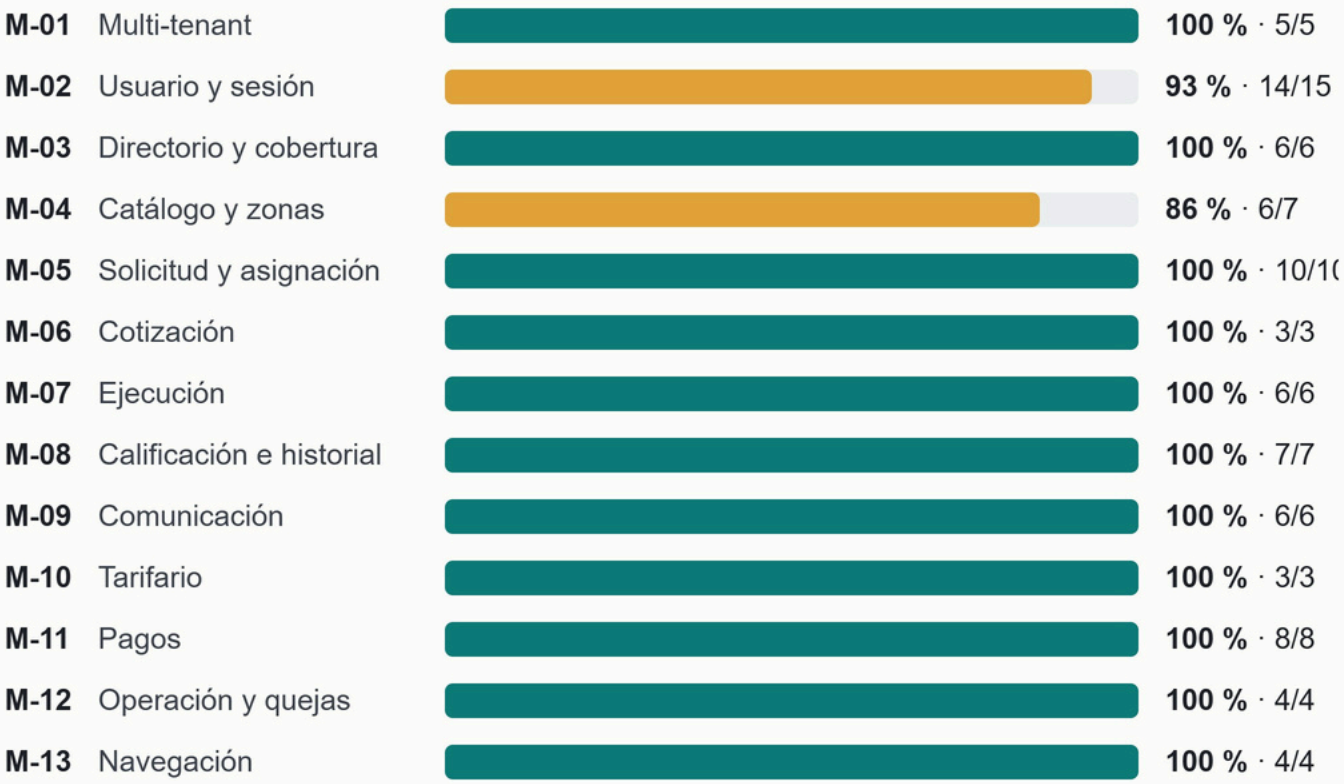
v1.0.4
en producción
7 repositorios etiquetados el 5/11

11 de 13
módulos al 100 %
M-02 y M-04 con una funcionalidad pendiente cada uno

MANI · TRAMA · Fuente: Jira, exportación del 5-nov-2026

DOCUMENTOS DE CIERRE · AVANCE

Avance por módulo al cierre



84

funcionalidades del plan

Comprometidas en el Sprint 5

97,6 %

finalizadas (82)

En Done en Jira

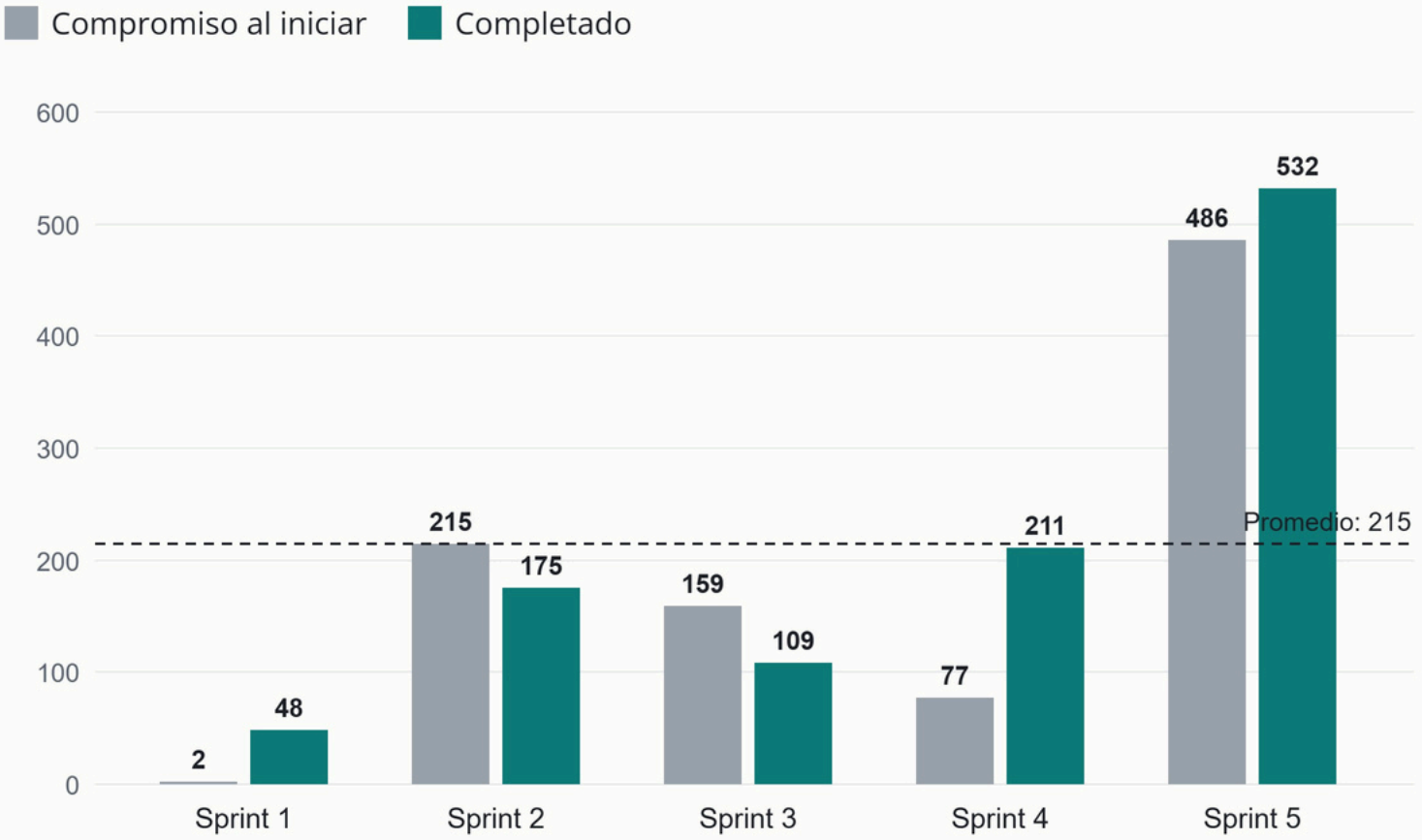
2

pendientes

1383 (cifrado) y HU-M04-05 (sin historia en Jira)

REPORTES DE JIRA · VELOCIDAD

Cada sprint entregamos más



532 SP

completados en el Sprint 5

Frente a 486 comprometidos al iniciar: 109 %

× 2,5

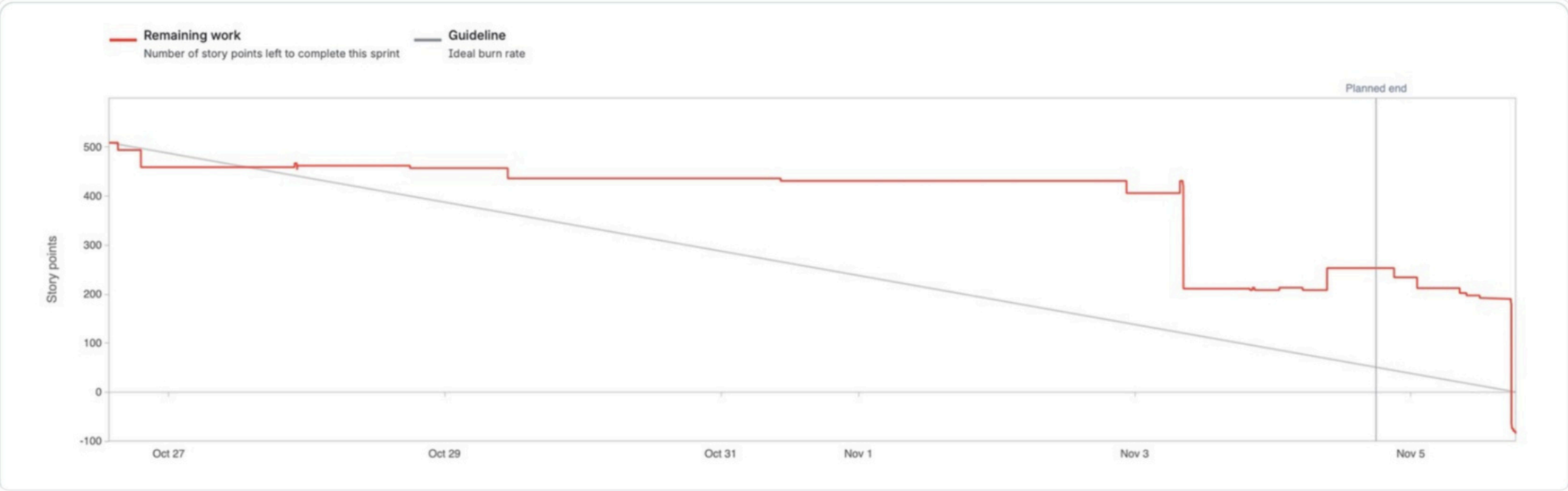
frente al Sprint 4

De 211 a 532 SP completados

MANI · TRAMA · Fuente: Jira, Velocity report

REPORTES DE JIRA · BURNDOWN

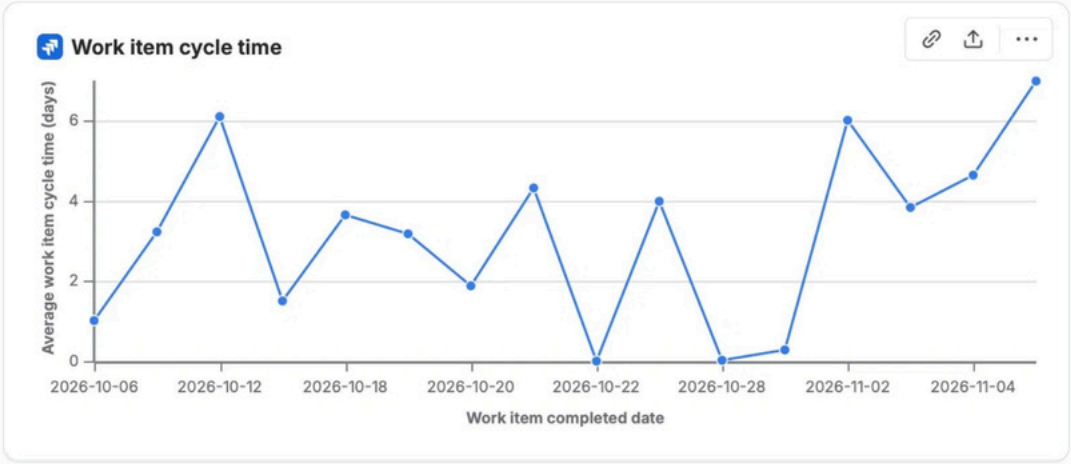
Cómo se quemó el sprint



Unos 510 SP al inicio y cierre con todo el alcance quemado. La quema se concentró en dos momentos: el 3/11 (validación de QA) y el 5/11 (cierre y despliegue).

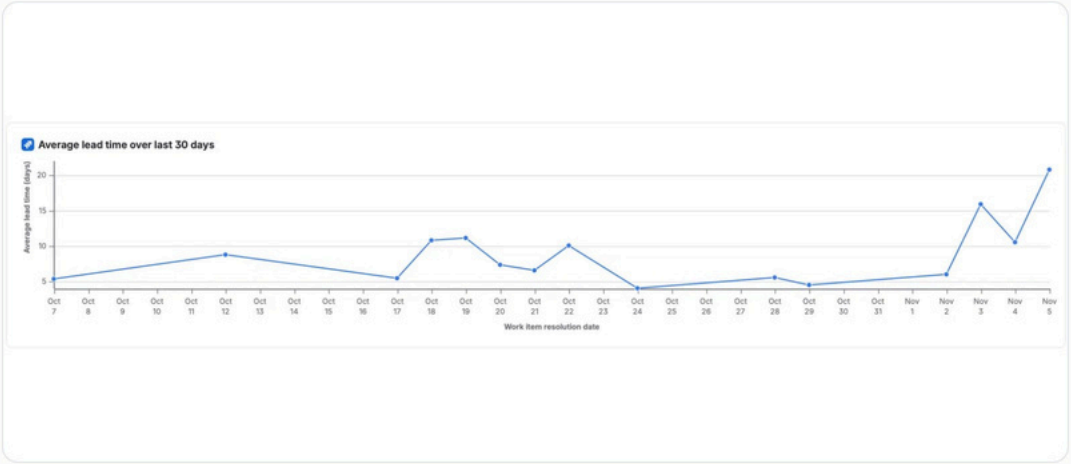
Cuánto tarda un ítem en cerrarse

Cycle time (de en curso a terminado)



Entre 0 y 4 días hasta el 30/10; sube a 4–7 días en el cierre porque los ítems esperaron la validación de QA.

Lead time (de creado a resuelto)



Entre 4 y 11 días en octubre; sube a 16–21 días al cierre porque se cerraron ítems creados semanas antes.

INDICADORES

Sprint 4 frente a Sprint 5

Indicador	Sprint 4	Sprint 5
Historias en Done	20 de 20	84 de 85
Story points completados	211 SP	532 de 537 SP
Predictibilidad (completado sobre lo del sprint)	94,6 %	99,1 %
Defectos	7 registrados, 4 corregidos	9 registrados, 9 corregidos
Despliegue	Solo QA	QA y producción (v1.0.4)
Cycle time p85	4,5 días	≈ 6 días
Ítems cerrados después del fin planeado	—	123 el 5/11

DOCUMENTOS DE CIERRE

Qué se entregó y qué quedó pendiente

Entregado en v1.0.4

- Multi-tenant con aislamiento entre empresas
- Directorio con KYC, cobertura y disponibilidad
- Solicitud y asignación sin doble asignación
- Cotización, ejecución, adiciones y pago sandbox
- Calificación, cierre, historial y avisos
- Panel del admin: categorías, tarifas, quejas y métricas

Pendiente con dueño

- 1383: cifrado de documento y teléfono
- Bugs de QA 1633 a 1637 (pago tras finalizar, foto, aliados por solicitud, estado en vivo, cliente-tenant)
- Rendimiento bajo carga: ampliar el pool de Core y volver a medir
- 901, 1418 y 1453: correcciones en release por comprobar en QA

Fuera del MVP

- Pagos reales con operador (hoy sandbox)
- Rastreo del técnico en vivo
- Kubernetes (hoy Docker Compose)
- ADR aún en estado Propuesto

DOCUMENTOS DE CIERRE · ACTA DE ACEPTACIÓN

Aceptado con condiciones

84 de 85
historias en Done
Validadas por QA y con video

53
pruebas de la PO desde la app
3 y 4 de noviembre, en QA

9 de 9
pasos del recorrido
Calificación y cierre conectados el 4-nov

5
bugs abiertos
Fuera del sprint, con descripción

Lo que acepto

- Ciclo completo del servicio en QA y en producción
- Aislamiento entre empresas y una sola asignación
- Pagos simulados con comprobante y sin doble pago
- Panel del administrador y del aliado

Condiciones

- Cifrado de datos sensibles (1383)
- Corregir los bugs 1633 a 1637
- Rendimiento bajo carga (QAS-03, 15 y 19)
- Validar los flujos en producción y firmar

Lecciones aprendidas

1. **Separar la lógica de la base desde el inicio**
   Migrar unas 32 funciones PL/pgSQL a servicios costó un sprint completo.

2. **Medir ayuda a mejorar**
   Con las métricas de Jira la predictibilidad pasó de 67,9 % a 94,6 %.

3. **Decidir por escrito**
   Los ADR dejaron claro el porqué de cada decisión y evitaron repetir discusiones.

4. **Hecho significa verificado**
   Una historia solo cuenta si QA la aprueba y el PO la valida desde la app, con evidencia.



Gracias

¿Preguntas?

TRAMA · La estructura detrás del servicio.